

0.23.1. Sans « x = » : la console range la position — de base, ailleurs

Le 0.23 sur la ligne 8, avec le B : toujours sans « x = ».

1. Ce qu'il faut comprendre

C'est le 0.23, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **ligne 8** pour la lettre, la **ligne 10** pour le nombre.

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.23.2 met les deux ensemble.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1639 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

2. UNE VARIABLE — X

```
uint8_t x = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    deplace_x(x, 8, ALPHABET[1], pas);    // sans « x = » : x vaut 5 quand même
    nombre(0, 10, x);                     // 005
    deplace_x(x, 8, ALPHABET[1], -pas);   // x vaut 0
    nombre(0, 10, x);                     // 000

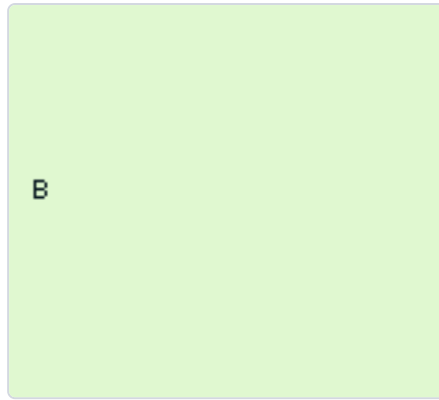
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

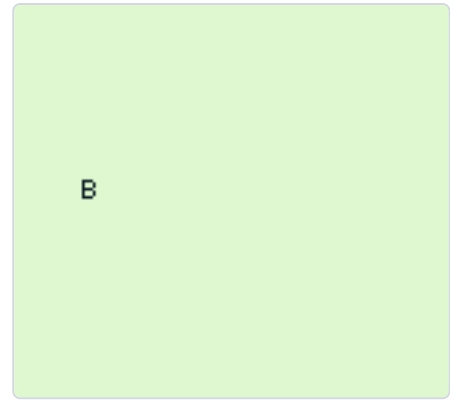
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le B va et revient sur la ligne 8 ; 005 puis 000 en dessous.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ sans « x = », x a reçu 5

✓ puis 0

Ensuite : **0.23.2. Sans « x = » : la console range la position — doublé, deux positions** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++